[Dat Nguyen] Process Fundamentals, Memory Layout

1. Program and Process

- **Program:** A saved file on the disk that contains code.
- **Process:** A program that is running on the computer after being loaded into RAM.

Workflow: Program on disk ➔ Load ➔ Memory ➔ CPU executes ➔ Process

2. PID and PPID (Parent Process ID)

2.1 PID (Process ID)

- At any given time, every running process has a unique PID so the OS can tell them apart. No two processes share the same PID at the same time.
- When a process ends, its PID is freed. The OS can give that PID to a new process later.

```
Syntax : getpid()
```

2.2 PPID (Parent Process ID)

PPID is the ID of the parent process that created the current process (child process).

- A child process gets a PPID that equals the parent process's PID.
- The child process inherits some resources from the parent process, but not all of them.

```
Syntax : getppid()
```

2.3 Examle:

```
int main(void)
{
    printf("My PID  = %d\n", getpid());
    printf("My PPID = %d\n", getppid());

    return 0;
}
```

Output:

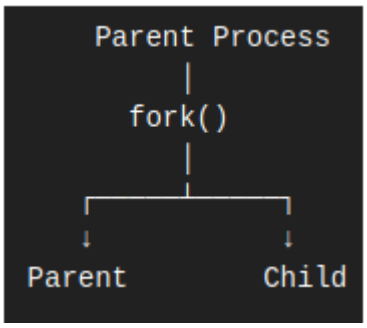
```
My PID  = 41436
My PPID = 29887
```

3. fork and vfork

3.1 fork

`fork()` is a system call used to create a new process.

- **Parent Process:** The process that calls `fork()`.
- **Child Process:** The new process that is created.
- When `fork()` is called, the OS creates a child process by copying the parent process using **Copy-on-Write**



`fork()` returns three possible values to help you handle the outcome:

- `> 0` **(Child's PID):** Returned to the **parent process**.
- `0`**:** Returned to the **child process**.
- `-1`**:** Returned to the parent if process creation **failed**.

```
int main() {
    printf("=========================================\n");
    printf("1. BEFORE FORK:\n");
    printf(" -> Current PID: %d\n", getpid());
```

```
    printf(" -> PPID (Terminal/Shell ID): %d\n", getppid());
    printf("========================================\n\n");

    // Call fork() to duplicate the process
    pid_t pid = fork();

    if (pid < 0) {
        printf("Failed to create child process!\n");
        return 1;
    }
    else if (pid == 0) {
        // --- CHILD PROCESS CODE ---
        // Sleep for 1s so the Parent output appears first for clarity
        sleep(1);
        printf("2. INSIDE CHILD PROCESS:\n");
        printf(" -> Child PID: %d\n", getpid());
        printf(" -> Child PPID (Parent's PID): %d\n\n", getppid());
    }
    else {
        // --- PARENT PROCESS CODE ---
        printf("3. INSIDE PARENT PROCESS:\n");
        printf(" -> Parent PID: %d\n", getpid());
        printf(" -> Newly created Child PID: %d\n\n", pid);

        // Parent calls wait() to wait for the child to finish and cleanup
        wait(NULL);
        printf("4. PARENT FINISHED REAPING CHILD (Cleaned up Zombie process).\n");
    }

    return 0;
}
```

Output:

```
1. BEFORE FORK:
 -> Current PID: 42140
 -> PPID (Terminal/Shell ID): 29887
========================================

3. INSIDE PARENT PROCESS:
```

```
 -> Parent PID: 42140
 -> Newly created Child PID: 42141

2. INSIDE CHILD PROCESS:
 -> Child PID: 42141
 -> Child PPID (Parent's PID): 42140

4. PARENT FINISHED REAPING CHILD (Cleaned up Zombie process).
```

3.2 vfork

`vfork()` is used to create a new process, but unlike `fork()`, it is optimized for fast execution with two main differences:

- **Parent process is suspended:** The parent process stops running until the child process calls `exec()` or `_exit()`.
- **Shared Memory:** The child and parent share the exact same memory space. Any changes made by the child directly modify the parent's variables.

Example 1: Parent Suspension

```
int main() {
    printf("[START] C program starts...\n");

    pid_t pid = vfork();

    if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("  [CHILD] I am created! Sleeping for 3 seconds...\n");
        sleep(3);
        printf("  [CHILD] Calling _exit() to exit and wake up Parent!\n");
        _exit(0); // MUST use _exit(), do not use exit() or return!
    }
    else if (pid > 0) {
        // --- PARENT PROCESS ---
        // This line only prints after the child sleeps for 3s and exits
        printf("[PARENT] I woke up after the Child exited!\n");
    }

    return 0;
}
```

Output:

```
START] C program starts...
[CHILD] I am created! Sleeping for 3 seconds...
[CHILD] Calling _exit() to exit and wake up Parent!
[PARENT] I woke up after the Child exited!
```

Example 2: Shared Memory

```
int main() {
    int count = 100; // Shared variable

    printf("Initial value of count = %d\n", count);

    pid_t pid = vfork();

    if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("  [CHILD] Changing count = 999\n");
        count = 999; // Directly modifies the parent's memory!
        _exit(0);
    }

    // --- PARENT PROCESS ---
    // Checking count after child exits
    printf("[PARENT] Current value of count = %d\n", count);
    // Prints: 999 (If standard fork() was used, this would still be 100)

    return 0;
}
```

Output:

```
Initial value of count = 100
[CHILD] Changing count = 999
[PARENT] Current value of count = 999
```

3.2 Compare fork and vfork

Criteria	`fork()`	`vfork()`

Memory Space	Creates a separate memory copy for the child (Copy-on-Write).	Child **SHARES** the exact same memory space (Stack, Heap, Data) with the parent.
Execution Order	Parent and child run **in parallel** (order is not guaranteed).	Parent is **BLOCKED** completely until the child calls `exec()` or `_exit()`.

4. Process Termination

Process termination means a process stops running and finishes its execution.

exit() vs _exit()

Feature	`exit()`	`_exit()`
Type	C Library function (`stdlib.h`)	System call (`unistd.h`)
Cleanup	Performs cleanup before exiting	Ends immediately without cleanup
Exit Handlers	Calls `atexit()` / `on_exit()` functions	Does not call `atexit()` / `on_exit()`
Stdio Buffers	Flushes I/O buffers (e.g., `printf`)	Does not flush I/O buffers

return vs exit()

Action	`return`	`exit()`
Control Flow	Returns control back to `main()` to finish remaining code	Immediately terminates the program
Execution Path	Continues execution in the caller function	Never returns to `main()` or caller function

5 wait(), waitpid(), and waitid() Functions

5.1 `wait()` Function

- **Process Suspension:** The parent process pauses execution until one of its child processes completely finishes.
- **Prevents Zombie Processes:** When a child terminates, the OS retains its PID and exit status in memory. Calling `wait()` cleans up the child's resources and releases its entry from the OS process table.

Examlple: forcing the child process to complete its output before the parent starts counting

```
int main() {
    int id = fork();
    int n;
    if(id == 0){
        n = 1;
    } else {
        n = 6;
    }

    if(id != 0){
        wait(NULL);
    }

    for(int i = n; i < n + 5 ; i++){
        printf("%d ", i);
        fflush(stdout);
    }
    printf("\n");
    return 0;
}
```

Output:

```
1 2 3 4 5
6 7 8 9 10
```

5.2 `waitpid()` Function

- **Similar to wait(), but waitpid() allows the parent process to wait for and collect the termination status of a specific child process by passing its PID.**

Prototype:

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- `options` **Parameter:**
  - `0`: Default blocking mode; suspends the parent process until the specified child process terminates.
  - `WNOHANG`: Non-blocking mode; returns `0` immediately if the specified child has not yet terminated, allowing the parent to continue execution.

Example 1: Option = 0

```
int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child Process
        printf("[Child - PID: %d] Running and working for 3 seconds...\n", getpid());
        sleep(3);
        exit(50); // Child exits with status 50
    }

    else {
        // Parent Process
        int status;
        printf("[Parent - PID: %d] Created child process (PID: %d). Waiting...\n", getpid(),
pid);

        pid_t wpid = waitpid(pid, &status, 0);

        if (wpid == -1) {
            perror("waitpid error");
            return 1;
        }

        // Check if child exited normally
        if (WIFEXITED(status)) {
            int exit_code = WEXITSTATUS(status);
            printf("[Parent] Child process (PID: %d) exited normally with code: %d\n", wpid,
exit_code);
        } else if (WIFSIGNALED(status)) {
            printf("[Parent] Child process terminated by signal: %d\n", WTERMSIG(status));
        }
    }
    return 0;
}
```

Output:

```
[Cha - PID: 7867] Đã tạo ti□n trình con (PID: 7868). B□t đ□u chờ...
[Con - PID: 7868] Đang chạy và làm việc trong 3 giây...
[Cha] Ti□n trình con (PID: 7868) đã k□t thúc bình thường với mã thoát: 50
```

Example 2: Option = WNOHANG

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child Process: Heavy processing work for 5 seconds
        printf("[Child] Processing heavy data for 5s...\n");
        sleep(5);
        printf("[Child] Completed work!\n");
        exit(100);
    }
    else if (pid > 0) {
        // Parent Process: Periodically checks child status using WNOHANG
        int status;
        pid_t result;

        while (1) {
            // Check child status without blocking
            result = waitpid(pid, &status, WNOHANG);

            if (result == 0) {
                // Child is still running -> Parent performs background tasks
                printf("[Parent] Child still running... Parent performing background work for
1s.\n");
                sleep(1);
            }
            else if (result == pid) {
                // Child finished -> Process exit code and break loop
                if (WIFEXITED(status)) {
                    printf("[Parent] Child finished with exit code: %d\n",
WEXITSTATUS(status));
                }
                break;
```

```
            }
            else {
                perror("waitpid error");
                break;
            }
        }
    }

    return 0;
}
```

Output:

```
[Child] Processing heavy data for 5s...
[Parent] Child still running... Parent performing background work for 1s.
[Parent] Child still running... Parent performing background work for 1s.
[Parent] Child still running... Parent performing background work for 1s.
[Parent] Child still running... Parent performing background work for 1s.
[Parent] Child still running... Parent performing background work for 1s.
[Child] Completed work!
[Parent] Child finished with exit code: 100
```

5.3 `waitid()` Function

Prototype:

```
waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

- **`idtype`** (**`idtype_t`**)**:** Defines the category of processes to wait for. Common options include:
  - `P_PID`: Wait for the child process whose PID matches `id`.
  - `P_PGID`: Wait for any child process whose Process Group ID (PGID) matches `id`.
  - `P_ALL`: Wait for any child process (the `id` argument is ignored).
- **`id`** (**`id_t`**)**:** The target ID (PID or PGID) associated with the specified `idtype`.
- **`infop`** (**`siginfo_t *`**)**:** Pointer to a `siginfo_t` structure where the kernel populates detailed exit/status metadata (e.g., `si_pid`, `si_status`, `si_code`).
- **`options`** (**`int`**)**:** Bitwise OR of flags specifying which state changes to wait for:
  - `WEXITED`: Wait for processes that have terminated.
  - `WSTOPPED`: Wait for processes that have been stopped by a signal.
  - `WCONTINUED`: Wait for processes that have been resumed by `SIGCONT`.
  - `WNOHANG`: Return immediately if no child has changed state.

```
int main() {
    pid_t pid_A = fork();

    if (pid_A == 0) {
        // --- CHILD A ---
        sleep(20);
        printf("[Child A] Finished after 20s\n");
        exit(1);
    }

    pid_t pid_B = fork();

    if (pid_B == 0) {
        // --- CHILD B ---
        sleep(1);
        printf("[Child B] Finished after 1s\n");
        exit(2);
    }

    // --- PARENT PROCESS ---
    siginfo_t info; // Replaces status int variable

    printf("[Parent - PID %d] Child A is %d, Child B is %d\n", getpid(), pid_A, pid_B);
    printf("[Parent] Waiting for Child A via waitid()... Ignoring Child B even if B finishes
first!\n");
    printf("[Parent] Blocking explicitly on Child A (PID: %d)...\n", pid_A);

    // Wait specifically for Child A using waitid
    // P_PID: Target specific PID
    // WEXITED: Wait for child process termination
    waitid(P_PID, pid_A, &info, WEXITED);

    printf("[Parent] Child A finished! (Exit code: %d)\n", info.si_status);
    printf("[Parent] Resting for 10s before cleaning up Child B...\n");

    sleep(10);

    // Clean up Child B which finished earlier using waitid
    waitid(P_PID, pid_B, &info, WEXITED);
    printf("[Parent] Child B finished! (Exit code: %d)\n", info.si_status);
```

```
    return 0;
}
```

Output:

```
[Parent - PID 11714] Child A is 11715, Child B is 11716
[Parent] Waiting for Child A via waitid()... Ignoring Child B even
if B finishes first!
[Parent] Blocking explicitly on Child A (PID: 11715)... [Child B]
Finished after 1s => B becomes a zombie process
dat        11715  0.0  0.0   2560   288 pts/1    S+   21:50   0:00 ./app
dat        11716  0.0  0.0      0     0 pts/1    Z+   21:50   0:00 [app] <defunct>
[Child A] Finished after 20s
[Parent] Child A finished! (Exit code: 1)
dat        11716  0.0  0.0      0     0 pts/1    Z+   21:50   0:00 [app] <defunct>
[Parent] Resting for 10s before cleaning up Child B...
[Parent] Child B finished! (Exit code: 2). Done!
```

5.4 Zombie vs orphan

- **Orphan Process:** Occurs when the parent process terminates before its child process. It is adopted by the init process (`PID 1`), which handles its cleanup safely upon completion.
- **Zombie Process:** Occurs when the child process terminates before its parent, but the parent fails to call 'wait'. Its entry remains in the OS process table, consuming system resources.

Feature	Orphan Process	Zombie Process
State	Parent dies first; child continues running.	Child dies; parent continues running without calling `wait()`.
Impact	Safe (Adopted and cleaned up by `PID 1`).	Harmful (Wastes space in the OS Process Table)

Example: Zombie (5.3 `waitid()` Function)

Example: Orphan

```
int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
```

```
    }

    if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("[CHILD - PID %d] Initial PPID: %d\n", getpid(), getppid());
        printf("[CHILD] Switching to run 'sleep 10' command via execvp...\n");

        // Call execvp to load the /bin/sleep program, completely replacing the current C code
        char *args[] = {"sleep", "10", NULL};
        execvp("sleep", args);

        // This line only runs if execvp fails
        perror("execvp failed");
        exit(1);
    }
    else {
        // --- PARENT PROCESS ---
        printf("[PARENT - PID %d] Preparing to exit early...\n", getpid());
        sleep(2); // Wait 2s to allow child time to call execvp
        printf("[PARENT] Parent has exited!\n");
        exit(0); // Parent exits, child (currently running 'sleep 10') becomes an orphan
    }

    return 0;
}
```

Output:

```
[PARENT] PID: 20899. Preparing to exit early...
[CHILD] Initial PID: 20900 | PPID (Parent): 20899
[CHILD] Sleeping for 10s to wait for Parent to exit first...
[PARENT] Parent has exited!
[CHILD] After becoming an orphan -> PID: 20900 | New PPID (Adoptive Parent): 2250
```

6. `system()` Function

- **Shell Command Execution:** The `system()` function passes a command string to the host environment's command processor (shell) to be executed.
- **Return Value:** Returns `0` (or the exit status returned by the shell) upon successful execution of the command.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    system("ls");
    system("echo 'some content' > text.txt");
    system("cat text.txt");

    if(system("ls") == 0){
        printf("Success executing command\n");
    }
    return 0;
}
```

Output:

```
app  Section2  Section3  test.c  text.txt
some content
app  Section2  Section3  test.c  text.txt
Success executing command
```

7. Process Groups, Sessions, Controlling Terminal & Job Control

7.1 Process Groups

- **Definition:** A Process Group is a collection of one or more processes sharing a unique Process Group ID (PGID).
- **Process Group Leader:** Each group has a leader process that creates the group; the PGID equals the leader's PID.
- **Signal Broadcast Purpose:** Allows the OS (and Shell/Terminal) to deliver signals to an entire group of processes simultaneously rather than target each process individually.
- **Pipeline Example:** Running `ls -l | grep "test" | wc -l` spawns three processes with distinct PIDs inside the same Process Group. Pressing `Ctrl + C` sends `SIGINT` to the entire PGID, terminating all three processes concurrently.

Key System Call: `setpgid()`

- **Syntax:** `int setpgid(pid_t pid, pid_t pgid);`

- **PID Parameter (`pid = 0`):** Refers to the calling process itself.
- **PGID Parameter (`pgid = 0`):** Sets the PGID equal to the target process's PID, making it the leader of a new process group.

```
int main() {
    // Set self as Group Leader (PGID = PID)
    setpgid(0, 0);
    pid_t leader_pid = getpid();

    printf("=== CHECK PID, PPID AND PGID ===\n");
    printf("[LEADER] PID: %d | PPID: %d | PGID: %d\n", getpid(), getppid(), getpgrp());

    if (fork() == 0) {
        // Child 2
        setpgid(0, leader_pid);
        printf("[CHILD 2] PID: %d | PPID: %d | PGID: %d\n", getpid(), getppid(), getpgrp());
        exit(0);
    }

    if (fork() == 0) {
        // Child 3
        setpgid(0, leader_pid);
        printf("[CHILD 3] PID: %d | PPID: %d | PGID: %d\n", getpid(), getppid(), getpgrp());
        exit(0);
    }

    wait(NULL);
    wait(NULL);
    return 0;
}
```

Output:

```
=== CHECK PID, PPID AND PGID ===
[LEADER] PID: 19613 | PPID: 10987 | PGID: 19613
[CHILD 2] PID: 19614 | PPID: 19613 | PGID: 19613
[CHILD 3] PID: 19615 | PPID: 19613 | PGID: 19613
```

7.2 Sessions

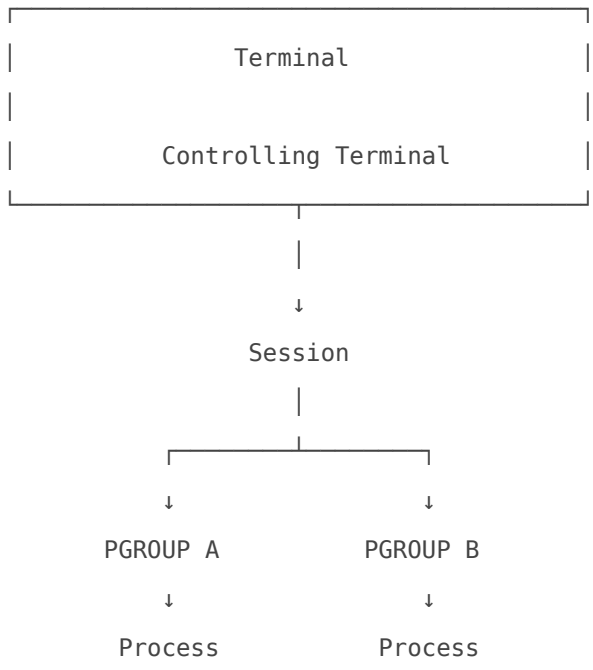
- **Definition:** A Session is a collection of one or more process groups.

- **Session ID (SID):** All processes within the same session share a unique identifier called a Session ID (SID).
- **Session Leader:** The process that creates the session is the Session Leader. Its PID becomes the SID for the entire session.

```
Session
   │
   ├── Process Group A
   │      ├── Process 1
   │      └── Process 2
   │
   └── Process Group B
          ├── Process 3
          └── Process 4
```

7.3 Controlling Terminal

- **Input and Signals:** The controlling terminal manages user input and signals (such as `SIGINT` from `Ctrl+C` or `SIGTSTP` from `Ctrl+Z`), forwarding them directly to the active foreground process group within the session.



7.5 Foreground and Background Process Groups

- **Foreground Process Group:**
  - Executes directly and takes control of the Controlling Terminal (TTY).
  - Only **one** Foreground Process Group can exist at any given time within a Session.

    - Directly receives keyboard input and control signals (such as `Ctrl+C` for `SIGINT` and `Ctrl+Z` for `SIGTSTP`).
- **Background Process Group:**
    - Executes in the background, releasing the terminal prompt for the user to enter further commands.
    - Multiple Background Process Groups can run concurrently in a single Session.
    - Cannot read input directly from the terminal keyboard (attempts to do so trigger a `SIGTTIN` signal).
    - Can be terminated directly in the background using `kill %<job_id>` or brought back to the foreground (`fg`) and stopped via `Ctrl+C`.

8. Command-Line Arguments and Environment List

8.1 Command-Line Arguments

- **Main Function Signature:** Handled via `int main(int argc, char *argv[])`.
- **Argument Count (`argc`):** Represents the total number of command-line arguments passed to the program, including the executable name itself.
- **Argument Vector (`argv`):** An array of string pointers (`char *[]`), where each element `argv[i]` points to the corresponding argument string.
- **Program Name Index (`argv[0]`):** Always stores the name (or path) of the executed program, which is why `argc` is always at least `1` even with no additional arguments passed.
- **Null Termination:** `argv[argc] == NULL` is required by standard specification to allow system calls or algorithms to identify the end of the argument array.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int i;

    // Print total number of command-line arguments
    printf("Number of arguments received (argc) = %d\n", argc);

    for (i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    // Verify specification: argv[argc] must be NULL
    if (argv[argc] == NULL) {
```

```
        printf("argv[argc] is NULL pointer (Valid according to standard).\n");
    }


    return 0;
```

Output:

```
Number of arguments received (argc) = 6
argv[0] = ./app
argv[1] = 1
argv[2] = 2
argv[3] = 3
argv[4] = 4
argv[5] = 5
argv[argc] is NULL pointer (Valid according to standard).
```

8.2 Environment List Management

- **Structure:** Each process maintains an environment list represented as an array of null-terminated strings in the key-value format `NAME=value`.
- **Inheritance:** A child process created via `fork()` inherits a copy of its parent's environment. Modifying environment variables in either process does not affect the other.
- **Global Access:** C programs access the environment list via the global variable `extern char **environ`, which is an array of string pointers terminated by a `NULL` pointer.
- **getenv:** Searches the environment list for `name` and returns a pointer to the value string, or `NULL` if not found.
- **setenv:** Adds or modifies an environment variable in the current process:
  - `overwrite = 1`: Overwrites the existing value if `name` already exists.
  - `overwrite = 0`: Retains the original value if `name` already exists.

```
extern char **environ;


int main() {
    // 1. Read an existing environment variable using getenv()
    char *user = getenv("USER");
    if (user != NULL) {
        printf("1. Current user (USER): %s\n", user);
    } else {
        printf("1. Environment variable USER not found.\n");
    }


    // 2. Add a new environment variable using setenv()
```

```
    // The third parameter (1) allows overwriting if the variable already exists
    if (setenv("MY_VAR", "Hello_Linux_System", 1) == -1) {
        perror("setenv error");
        return EXIT_FAILURE;
    }
    printf("2. Set MY_VAR = %s\n", getenv("MY_VAR"));

    // 3. Remove the environment variable using unsetenv()
    if (unsetenv("MY_VAR") == -1) {
        perror("unsetenv error");
        return EXIT_FAILURE;
    }

    char *check_var = getenv("MY_VAR");
    printf("3. After deletion, MY_VAR value: %s\n",
           (check_var == NULL) ? "NULL (Does not exist)" : check_var);

    // 4. Iterate through the complete environment list
    char **ep;
    int count = 0;
    printf("\n=== Full Environment List ===\n");
    for (ep = environ; *ep != NULL; ep++) {
        printf("[%d] %s\n", count, *ep);
        count++;
    }

    return 0;
}
```

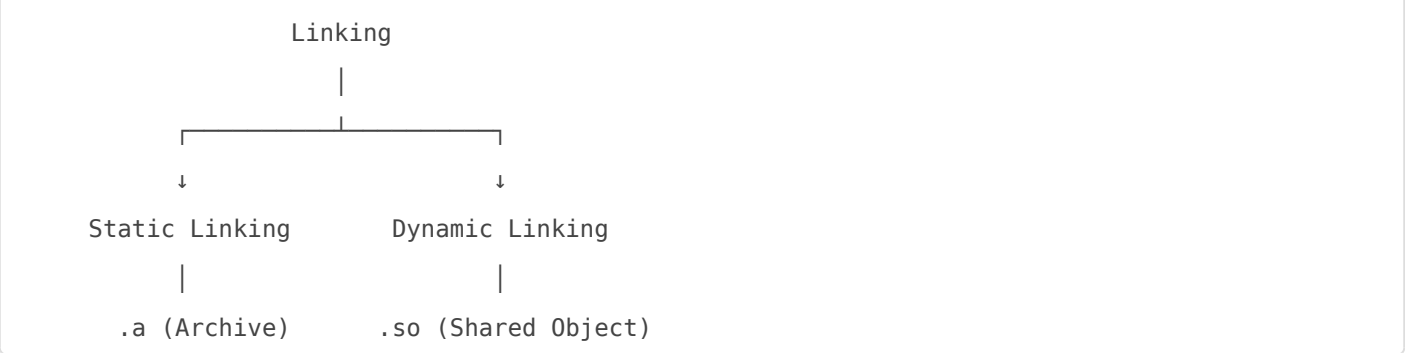
Output

```
dat@dat-asus:~/Documents/Zhone-Training$ ./app
1. Current user (USER): dat
2. Set MY_VAR = Hello_Linux_System
3. After deletion, MY_VAR value: NULL (Does not exist)

=== Full Environment List ===
[0] SHELL=/bin/bash
[1] SESSION_MANAGER=local/dat-asus:@/tmp/.ICE-unix/2596,unix/dat-asus:/tmp/.ICE-unix/2596
[2] QT_ACCESSIBILITY=1
[3] COLORTERM=truecolor
[4] XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
[5] XDG_MENU_PREFIX=gnome-
[6] TERM_PROGRAM_VERSION=1.134.0
[7] XDG_CONFIG_DIRS_VSCODE_SNAP_ORIG=/etc/xdg/xdg-ubuntu:/etc/xdg
[8] GNOME_DESKTOP_SESSION_ID=this-is-deprecated
[9] GNOME_KEYRING_CONTROL=/run/user/1000/keyring
[10] GDK_BACKEND_VSCODE_SNAP_ORIG=
[11] FONTCONFIG_PATH=/etc/fonts
[12] GNOME_SHELL_SESSION_MODE=ubuntu
[13] GIO_MODULE_DIR_VSCODE_SNAP_ORIG=
```

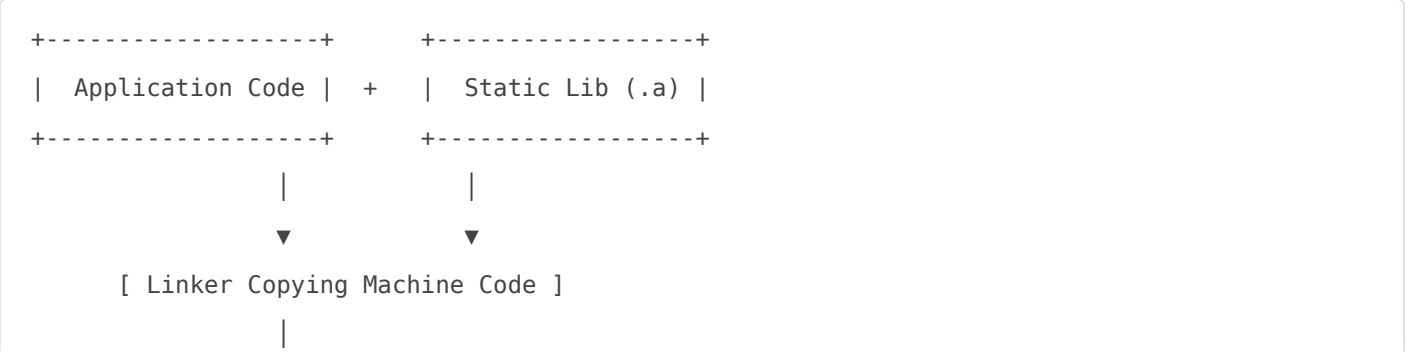
9. Static vs Dynamic Linking

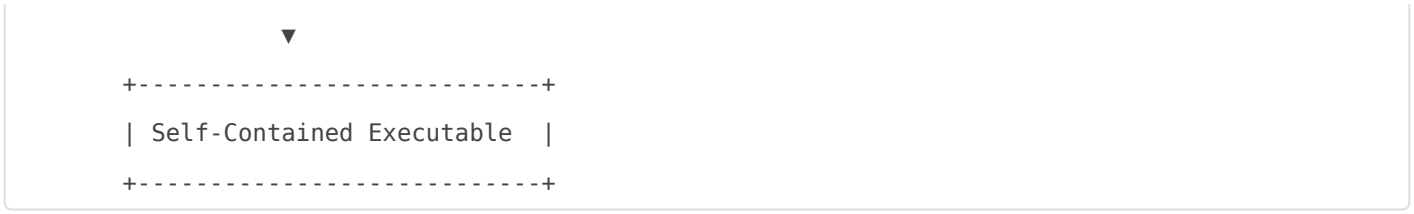
A library is a compiled collection of reusable code functions and definitions that can be linked into applications.



9.1 Static Linking (`.a`)

Static libraries are archives containing object files (`.o`). During the compilation phase, the linker extracts the required machine code modules from the `.a` file and copies them directly into the final executable binary.

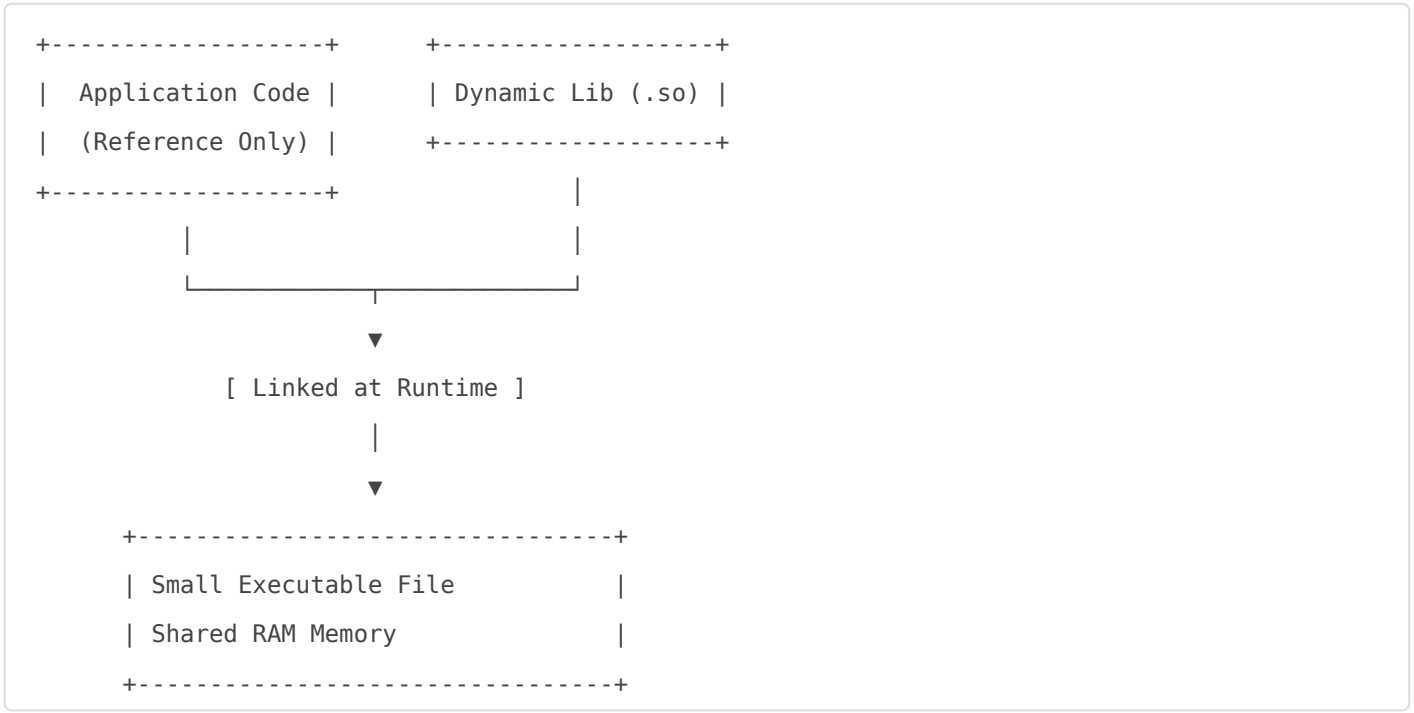




- **Binary Size:** The executable file is large because library code is copied inside. Multiple running apps waste disk space and RAM.
- **Portability:** Easy to run anywhere. The file runs on its own without needing extra library files.
- **Maintenance:** Updating library code requires rebuilding (recompiling) the entire app.

9.2 Dynamic Linking (`.so`)

Dynamic libraries exist as independent shared object (`.so`) files on the file system. During compilation, the linker does not copy the library's machine code into the application executable, but instead records symbolic references and addresses. The actual linking process is deferred until the application is launched at runtime.



- **Saves Memory:** Executable files are small. Multiple running programs can share one library copy in RAM.
- **Easy Maintenance:** When fixing library bugs, you only recompile the `.so` file. The main program runs normally without recompiling.
- **Requires Library File:** The application cannot start if the `.so` file is missing or in the wrong folder.

9.3 Compare Stactic vs Dynamic

Feature	Static Library (`.a`)	Shared / Dynamic Library (`.so`)
Linking Time	**Build time** (during compilation).	**Runtime** (when the program runs).
Code Embedding	Copies machine code directly into executable.	Stores only reference addresses to `.so` file.
Executable Size	Large (grows with library size).	Very small and lightweight.
RAM Usage	Higher (each app keeps a separate code copy).	Saves RAM (apps share one loaded `.so` copy).
Library Updates	Must recompile the entire app.	Recompile only `.so` file; no app recompile needed.
File Dependency	Independent (deleting `.a` does not affect app).	Dependent (missing `.so` file causes app failure).

10. `getrlimit` and `setrlimit` Functions (Resource Limits)

10.1 Resource Limits

Linux sets limits on each process so one program cannot use up all system resources. Each resource is controlled by two limits inside `struct rlimit`:

```
struct rlimit {
    rlim_t rlim_cur;  /* Soft limit: Current limit */
    rlim_t rlim_max;  /* Hard limit: Maximum ceiling */
};
```

10.2 getrlimit

Used to check current resource limits set for the process.

Prototype:

```
int getrlimit(int resource, struct rlimit *rlim);
```

10.3 setrlimit()

Used to change the resource limits for the calling process. Protopyte:

```
int setrlimit(int resource, const struct rlimit *rlim);
```

Example

```
void print_nofile_limit() {
    struct rlimit rl;
    if (getrlimit(RLIMIT_NOFILE, &rl) == 0) {
        printf("File Descriptors Limit -> Soft: %ld | Hard: %ld\n",
               (long)rl.rlim_cur, (long)rl.rlim_max);
    } else {
        perror("getrlimit error");
    }
}

int main() {
    struct rlimit new_rl;

    printf("--- Initial State ---\n");
    print_nofile_limit();

    // Set new soft limit to 2048 and hard limit to 4096
    new_rl.rlim_cur = 2048;
    new_rl.rlim_max = 4096;

    if (setrlimit(RLIMIT_NOFILE, &new_rl) == -1) {
        perror("setrlimit failed");
        exit(EXIT_FAILURE);
    }

    printf("\n--- State After setrlimit() ---\n");
    print_nofile_limit();

    return 0;
}
```

Output:

```
--- Initial State ---
File Descriptors Limit -> Soft: 1048576 | Hard: 1048576

--- State After setrlimit() ---
File Descriptors Limit -> Soft: 2048 | Hard: 4096
```

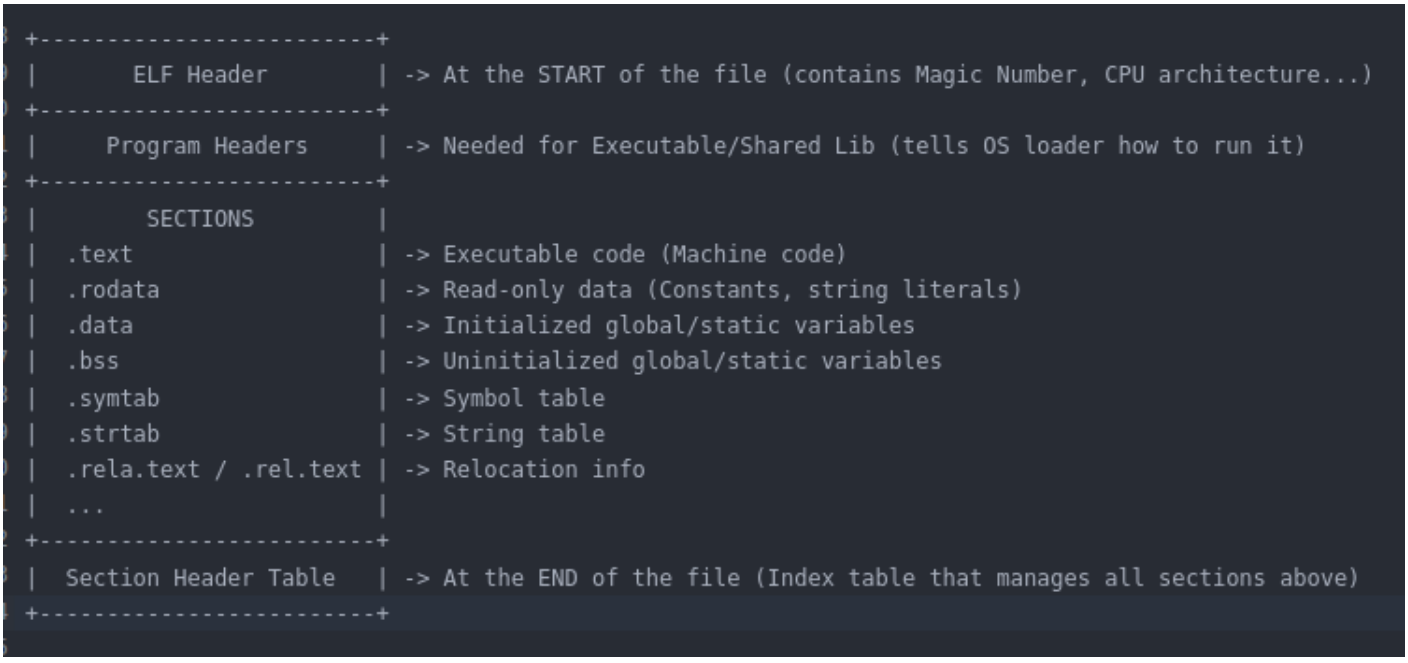
11. ELF Format, Symbol Table, Relocation - Supplementary Linking Concepts

11.1 ELF (Executable and Linkable Format)

ELF is the standard binary file format used in Linux for executable files, object files, and shared libraries.

```
Source Code (.c)
       |
       ▼ (gcc)
 Object File (.o)      ──► Relocatable ELF
 Shared Library (.so)  ──► Shared ELF
 Executable (app)      ──► Executable ELF
```

Structure:



11.2 Symbol table

Simple Summary

A **Symbol Table** is a crucial data structure inside an ELF binary file. It acts as a directory of names, storing detailed information about functions and global/static variables used in the program so that the linker and debugger can locate them in memory.

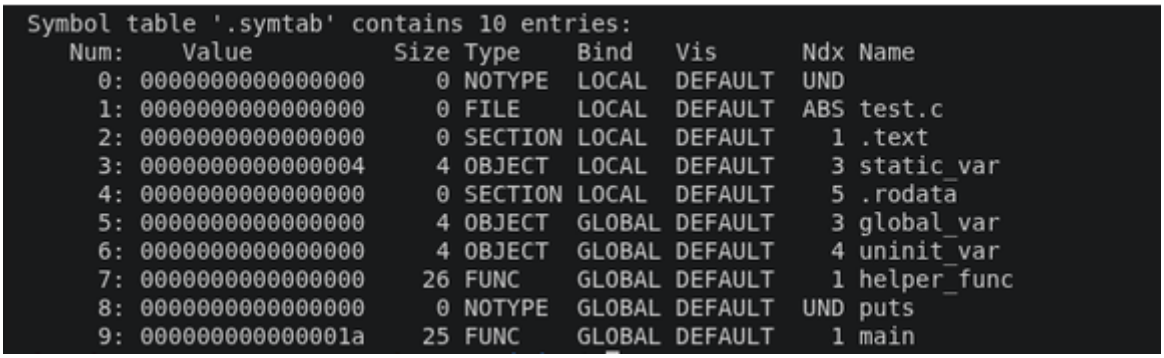
Code Example

```
int global_var = 100;       // GLOBAL OBJECT (.data section)
int uninit_var;             // GLOBAL OBJECT (.bss / COMMON section)
static int static_var = 50; // LOCAL OBJECT (.data section)

void helper_func() {        // GLOBAL FUNCTION (.text section)
    printf("Helper\n");
}

int main() {                // GLOBAL FUNCTION (.text section)
    helper_func();
    return 0;
}
```

Output:



```
Symbol table '.symtab' contains 10 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS test.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1 .text
     3: 0000000000000004     4 OBJECT  LOCAL  DEFAULT    3 static_var
     4: 0000000000000000     0 SECTION LOCAL  DEFAULT    5 .rodata
     5: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    3 global_var
     6: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    4 uninit_var
     7: 0000000000000000    26 FUNC    GLOBAL DEFAULT    1 helper_func
     8: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND puts
     9: 000000000000001a    25 FUNC    GLOBAL DEFAULT    1 main
```

11.3 Relocation

When compiling a source code file (e.g., main.c) into an object file (main.o), the Compiler cannot know the memory address of functions like printf or global variables defined in other files (like global_var).

=> As a result, the Compiler inserts temporary (dummy) addresses.

Example: We two files: foo.c and main.c.

foo.c

```
int my_var = 42;
long long my_var1 = 50;
void foo() {}
```

Compile foo.c into foo.o (an object file containing machine code).

Use nm foo.o to read and display the list of Symbols (Symbol Table) inside foo.o:

Output:

```
0000000000000000 T foo       (.text)
0000000000000000 D my_var□ (.data)□
0000000000000008 D my_var1□ (.data)
```

main.c

```
extern int my_var;
extern void foo();

int main() {
    foo();
    return my_var;
}
```

Compile main.c into main.o.

Run nm main.o:

Output:

```
                 U foo               (undefined because it is in another file)
0000000000000000 T main              (.text section)
                 U my_var            (undefined)
```

Linking step:

gcc main.o foo.o -o app

At this stage, the Linker merges foo.o and main.o:

```
Merges the .text sections of both files together.

Appends the .data section from foo.o.

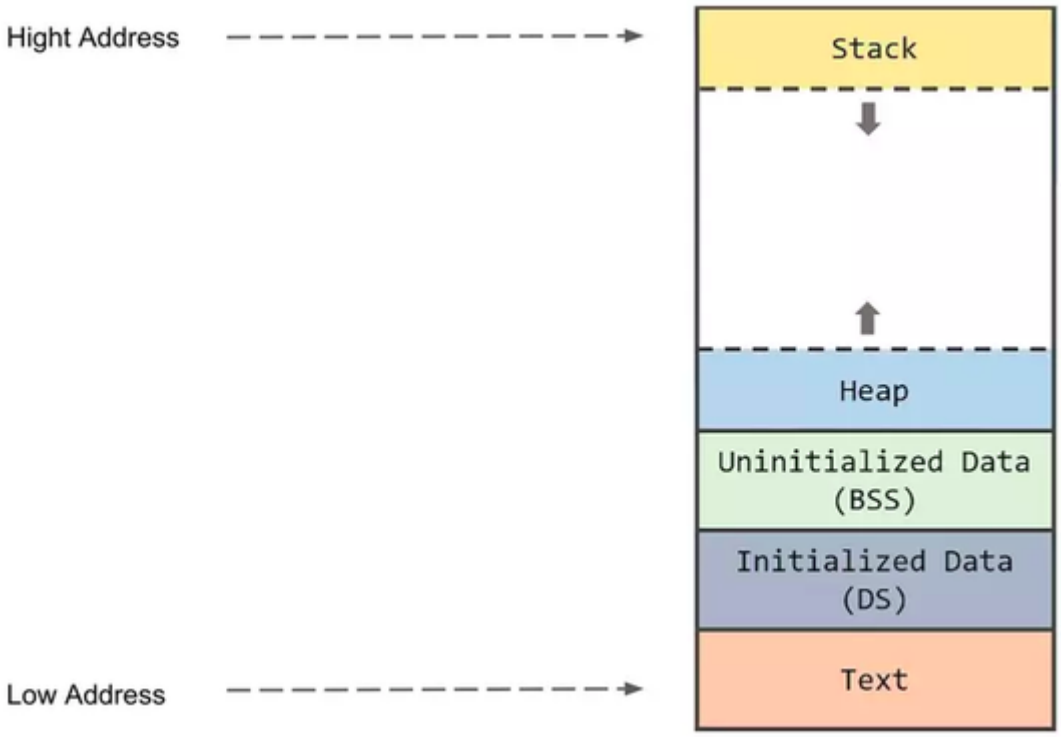
Replaces the two U (undefined) symbols in main.o with the actual merged memory addresses of
foo and my_var.
```

Output:

```
0000000000001129 T main
0000000000001143 T foo
0000000000004010 D my_var
0000000000004018 D my_var1
```

These are virtual memory addresses. When app runs, the Operating System's memory manager works together with the MMU (Memory Management Unit) to map these virtual addresses to actual physical addresses on RAM.

12. Layout memory



Segment	Purpose
Text Section	Stores compiled machine code (read-only).
Data Section	Stores initialized global and static variables.
BSS Section	Stores uninitialized global and static variables (set to 0).
Heap Section	Allocated at runtime via dynamic memory functions.
Stack Section	Stores local variables, function arguments, and return addresses.

```
int global_init_var = 100; // 1. Initialized global variable -> .data segment

// 2. Uninitialized global variable -> .bss segment
int global_uninit_var;
```

```
int main() {
    // 3. Local variable -> Stack segment
    int stack_var = 10;

    // 4. Dynamic memory allocation -> Heap segment
    int *heap_ptr = (int *)malloc(sizeof(int));

    printf("=== MEMORY REGIONS ADDRESSES ===\n");
    printf(".text  (code)         : %p\n", (void *)main);
    printf(".data  (initialized)  : %p\n", (void *)&global_init_var);
    printf(".bss   (uninitialized): %p\n", (void *)&global_uninit_var);
    printf("Heap   (dynamic alloc): %p\n", (void *)heap_ptr);
    printf("Stack  (local var)    : %p\n", (void *)&stack_var);

    // Free Heap memory after use
    free(heap_ptr);

    while(1);
    return 0;
}
```

output:

```
=== MEMORY REGIONS ADDRESSES ===
.text  (code)         : 0x6108dc5391a9
.data  (initialized)  : 0x6108dc53c010
.bss   (uninitialized): 0x6108dc53c018
Heap   (dynamic alloc): 0x61091b4632a0
Stack  (local var)    : 0x7fff0a7d282c

7fff0a7b4000-7fff0a7d6000 rw-p 00000000 00:00 0                          [stack]
61091b463000-61091b484000 rw-p 00000000 00:00 0                           [heap]
```

13 Stack Frame

13.1 Concept

Every time a function is called, it usually requires a memory region on the stack to store information necessary for function execution.

- `%rsp` **(Stack Pointer Register):** Contains the address of the current position on the stack. It points to the current top of the stack.
- `%rbp` **(Frame Pointer Register):** It can be used as a base reference point (base pointer) for the current stack frame.
- **Prologue:** The initialization phase of a stack frame.
- **Epilogue:** When a function finishes execution, it must restore the previous stack frame state.
- `ret`**:** When a function is called, the CPU pushes the return address onto the stack. Upon executing `ret`, the CPU pops the return address from the stack and jumps back to it.

13.2 Passing Arguments via Registers

Simple Summary

When a function takes arguments (inputs), the CPU passes the first **6 arguments using Registers** instead of Memory (RAM). If there are more than 6 arguments, the rest are placed onto the **Stack**.

Argument Register Order (x86-64 System V ABI)

```
int func(int a, int b, int c, int d,
             int e, int f, int g, int h)


a → RDI
b → RSI
c → RDX
d → RCX
e → R8
f → R9
g → stack
h → stack
```

Why Use Registers Instead of Memory/RAM?

- **Speed & Frequency:** Registers run at the exact same clock speed as the CPU execution engine. Reading or writing to a register happens almost instantaneously within 1 clock cycle.
- **Zero Bus Delay:** RAM is physically separated from the CPU chip. Requesting data from RAM requires signals to travel across physical circuit traces (the system bus), introducing memory latency and electrical propagation delays.

13.3 128-byte Red Zone

Simple Summary

The **Red Zone** is a reserved 128-byte memory space located immediately below the current stack pointer (`%rsp`). Functions can temporarily store local data in this region without spending extra CPU instructions to adjust (subtract from) `%rsp`.

Core Concepts

- **Definition:** A fixed 128-byte area directly below `%rsp` (`%rsp - 1` to `%rsp - 128`) reserved by the x86-64 System V ABI.
- **Primary Use Case:** Used by **Leaf Functions** (functions that do not call any other functions). Because a leaf function will not invoke child calls that push new data onto the stack,

code:

```
code:
int calculate_sum() {
    int a = 10;
    int b = 20;
    int c = a + b;
    return c;
}
```

zed_zone:

mov rbp, rsp ; rbp == rsp

mov DWORD PTR -12[rbp], 10 ; Write variable 'a' directly below %rsp

mov DWORD PTR -8[rbp], 20 ; Write variable 'b' directly below %rsp

mov DWORD PTR -4[rbp], eax ; Write variable 'c' directly below %rsp

```
                ┌──────────────────────┐
                │ Old RBP              │
 %rbp, %rsp ──>├──────────────────────┤
                │ -4[rbp]  (result)    │  Red Zone
                │ -8[rbp]  (var b)     │ (Written directly below %rsp,
                │ -12[rbp] (var a)     │  NO NEED for `sub rsp` / `leave`)
                └──────────────────────┘
```

no zed_zone

mov rbp, rsp

sub rsp, 16 ; Move %rsp down by 16 bytes

```
                ┌──────────────────────────┐
                │ Old RBP                  │
      %rbp ──>  ├──────────────────────────┤
                │ -4[rbp]  (result)        │ ─── Official Stack Frame
                │ -8[rbp]  (var b)         │     (Protected by the
                │ -12[rbp] (var a)         │      `sub rsp, 16` instruction)
      %rsp ──>  ├──────────────────────────┤
                └──────────────────────────┘
```

14. Reading compiler-generated Assembly (gcc -S / objdump -d) to identify a function's own stack frame

code:

```
#include <stdio.h>



int add(int a, int b){
   int c = 30;
   int res = a + b + c;
   return res;
}



int main(){
   int a = 10;
   int b = 20;
   int res = add(a, b);
   printf("Result = %d\n", res);
}
```

cc -S test.c => test.s(Assembly code)

```
main:
.LFB1:
    .cfi_startproc
    endbr64
    pushq   %rbp                        // Save the base pointer value
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp                  // Set base pointer = stack pointer => Create new base
pointer for main
    .cfi_def_cfa_register 6
    subq    $16, %rsp                   // Move stack pointer down by 16 bytes to store values
    movl    $10, -12(%rbp)              // Save value 10 at position rbp - 12
    movl    $20, -8(%rbp)               // Save value 20 at position rbp - 8
    movl    -8(%rbp), %edx              // Load value 20 into register edx
    movl    -12(%rbp), %eax             // Load value 10 into register eax
    movl    %edx, %esi                  // Copy value of register edx to esi => edx = esi = 20
    movl    %eax, %edi                  // Copy value of register eax to edi => eax = edi = 10
    call    add                         // Push Return Address to Stack, and jump to the add
function
    movl    %eax, -4(%rbp)              // Store register value eax = 60 into rbp - 4
    movl    -4(%rbp), %eax              // Load value 60 from rbp - 4 into register eax
    movl    %eax, %esi                  // Copy value of register eax to register esi => esi =
eax = 60
    leaq    .LC0(%rip), %rax            // Get pointer address of the string
    movq    %rax, %rdi                  // Copy string address in register rax to register rdi
    movl    $0, %eax                    // printf function has 0 floating-point arguments
    call    printf@PLT
    movl    $0, %eax                    // return 0
    leave
    .cfi_def_cfa 7, 8
    ret



add:
.LFB0:
    .cfi_startproc
    endbr64
    pushq   %rbp                        // Save the base pointer value of main
    .cfi_def_cfa_offset 16
```

```
    .cfi_offset 6, -16
    movq    %rsp, %rbp                  // Create base pointer for the function: rbp = rsp
    .cfi_def_cfa_register 6
    movl    %edi, -20(%rbp)             // Store register edi = 10 into red zone at rbp - 20
    movl    %esi, -24(%rbp)             // Store register esi = 20 into red zone at rbp - 24
    movl    $30, -8(%rbp)               // Store value 30 into red zone at rbp - 8
    movl    -20(%rbp), %edx             // Load value 10 from rbp - 20 into register edx
    movl    -24(%rbp), %eax             // Load value 20 from rbp - 24 into register eax
    addl    %eax, %edx                  // edx = eax + edx = 10 + 20 = 30
    movl    -8(%rbp), %eax              // Load value 30 from rbp - 8 into register eax
    addl    %edx, %eax                  // eax = eax + edx = 30 + 30 = 60
    movl    %eax, -4(%rbp)              // Store register value eax = 60 into red zone at rbp
- 4
    movl    -4(%rbp), %eax              // Load value 60 from red zone rbp - 4 into register
eax for return
    popq    %rbp                        // Pop old RBP back into %rbp (Restore main's Base
Pointer)
    .cfi_def_cfa 7, 8
    ret                                 // Pop Return Address into %rip to jump back to main
```

LAB

LAB1: Write a program using fork/exec/wait/waitpid; observe zombie and orphan processes

Zombie: 5.3 `waitid()` Function

Orphan: 5.4 Zombie vs orphan -> example orphan

LAB2: Reproduce a race condition between fork and exec, explain it, and fix it

Parent-Child Process Synchronization Race Condition

```
int main() {
    pid_t pid = fork();

    if (pid < 0) {
```

```
        perror("Fork failed");
        exit(1);
    }

    if (pid == 0) {
        // CHILD PROCESS
        // Simulate child process taking time before exec
        //sleep(1);

        // Change child process to run 'ls' command
        char *args[] = {"/bin/ls", "-l", NULL};
        execvp(args[0], args);

        perror("Exec failed");
        exit(1);
    } else {
        // PARENT PROCESS
        // Parent thinks child runs exec right away
        printf("[Parent] Forked child with PID: %d\n", pid);
        // RACE CONDITION: Parent checks child file /proc/<pid>/cmdline
        // If child has not run exec, cmdline still has the old program name (current program)
        char path[64];
        snprintf(path, sizeof(path), "/proc/%d/cmdline", pid);

        FILE *f = fopen(path, "r");
        if (f) {
            char buf[256] = {0};
            fread(buf, 1, sizeof(buf) - 1, f);
            fclose(f);
            printf("[Parent] Running child program name is: %s\n", buf);
        }

        // Wait for child to finish
        wait(NULL);
    }
    return 0;
}
```

output:

```
dat@dat-asus:~/Documents/Zhone-Training$ ./test
[Parent] Forked child with PID: 22864
[Parent] Running child program name is: ./test
total 28
drwxrwxr-x 5 dat dat  4096 Aug 20 17:24 Section2
drwxrwxr-x 2 dat dat  4096 Aug 23 20:49 Section3
-rwxrwxr-x 1 dat dat 16376 Aug 26 13:58 test
-rw-rw-r-- 1 dat dat  1385 Aug 26 13:58 test.c
```

Error Mechanism Explanation:

1. When `fork()` runs, the child process is an exact copy of the parent process. At this moment, both parent and child are running the same original executable code.
2. Calling `exec()` is asynchronous relative to the parent process. There is no guarantee for the parent process that the child process will run the `exec()` command immediately.
3. The parent process reads `/proc/<pid>/cmdline` right after `fork()`. Error Result: The parent reads the name of the current code file itself (`./test`) because the child process has not run `execvp()` yet. Expected Result: The parent wants to read `/bin/ls`.

Fix the Error: To fix the race condition between `fork()` and `exec()`, we need a synchronization mechanism between the parent and child processes. The standard and simplest way is to use a Pipe so the child process can tell the parent process right when it is ready or completes `exec()`.

```
int main() {
    int pfd[2];
    if (pipe(pfd) < 0) {
        perror("Pipe creation failed");
        exit(1);
    }

    // Set FD_CLOEXEC flag for the write end of the pipe
    // When exec() succeeds, this write end will close automatically
    fcntl(pfd[1], F_SETFD, FD_CLOEXEC);

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid == 0) {
```

```
    // CHILD PROCESS
    close(pfd[0]); // Close unused read end

    sleep(1); // Simulate work before exec

    char *args[] = {"/bin/ls", "-l", NULL};
    execvp(args[0], args);

    // Runs here only if exec fails
    perror("Exec failed");
    write(pfd[1], "F", 1); // Send error to parent
    exit(1);
} else {
    // PARENT PROCESS
    close(pfd[1]); // Close unused write end

    // SYNCHRONIZATION: Parent waits to read from pipe.
    // Read will block until child calls exec() successfully (pipe closes)
    // or child writes data to pipe (if exec fails).
    char dummy;
    read(pfd[0], &dummy, 1);
    close(pfd[0]);

    printf("[Parent] Confirmed child finished exec. PID: %d\n", pid);

    // Read child cmdline again -> Now it is surely /bin/ls
    char path[64];
    snprintf(path, sizeof(path), "/proc/%d/cmdline", pid);

    FILE *f = fopen(path, "r");
    if (f) {
        char buf[256] = {0};
        fread(buf, 1, sizeof(buf) - 1, f);
        fclose(f);
        printf("[Parent] Running child program name is: %s\n", buf);
    }

    wait(NULL);
}
```

```
    return 0;
}
```

output:

```
dat@dat-asus:~/Documents/Zhone-Training$ ./test
[Parent] Confirmed child finished exec. PID: 23322
[Parent] Running child program name is: /bin/ls
total 32
drwxrwxr-x 5 dat dat  4096 Aug 20 17:24 Section2
drwxrwxr-x 2 dat dat  4096 Aug 23 20:49 Section3
-rwxrwxr-x 1 dat dat 16592 Aug 26 14:11 test
-rw-rw-r-- 1 dat dat  1890 Aug 26 14:11 test.c
```

LAB3: Use ps -o pid,ppid,pgid,sid to observe the process group/session of the program just written

```
int main() {
    // Create child process
    if (fork() == 0) {
        // This is the Child Process
        printf("[Child] PID: %d, PPID: %d\n", getpid(), getppid());
    } else {
        // This is the Parent Process
        printf("[Parent] PID: %d, PPID: %d\n", getpid(), getppid());
    }

    // Keep both processes running for 60s to observe
    sleep(60);
    return 0;
}
```

output:

```
  PID        PPID    PGID     SID COMMAND
  35121    7869   35121     7869 test
  35122   35121   35121    7869 test
```

Analysis of Each Process:

Parent Process (PID 35121)

- **PID = 35121:** Process ID of the parent process.
- **PPID = 7869:** Parent process ID that created it is 7869 (this is the Bash/Zsh shell of your active terminal).
- **PGID = 35121:** The parent process is the leader of this Process Group because its PGID matches its PID.
- **SID = 7869:** It belongs to the current terminal session.

Child Process (PID 35122)

- **PID = 35122:** Unique process ID of the child process.
- **PPID = 35121:** The parent process that created it.
- **PGID = 35121:** It inherits and belongs to the same Process Group as the parent process.
- **SID = 7869:** It still belongs to the same terminal session.

LAB4: Write a program that prints the addresses of its own memory regions (text/data/bss/heap/stack)

12. Layout memory

LAB5: Use nm/readelf/ldd/objdump to inspect a self-written binary and produce a short report

```
#include <stdio.h>

int global_var = 100;       // Initialized global variable (.data)
int uninit_var;             // Uninitialized global variable (.bss)

static void secret_func() { // Static function (local symbol)
    printf("This is an internal function!\n");
}

int add(int a, int b) {     // Global function (global symbol)
    return a + b;
}

int main() {
    int x = 5, y = 10;
    secret_func();
    printf("Sum %d + %d = %d\n", x, y, add(x, y));
    return 0;
}
```

lld: Check shared libraries (Dynamic Libraries)

```
dat@dat-asus:~/Documents/Zhone-Training$ ldd test
        linux-vdso.so.1 (0x00007a7559047000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007a7558e00000)
        /lib64/ld-linux-x86-64.so.2 (0x00007a7559049000)
```

`libc.so.6`: Standard C library (contains functions like `printf`). `ld-linux-x86-64.so.2`: Linux dynamic linker/loader.

nm: Check the Symbol Table Lists function names, global variables, and their memory addresses.

```
dat@dat-asus:~/Documents/Zhone-Training$ nm test | grep -E "global_var|uninit_var|add|func|main"
0000000000001183 T add
0000000000001169 t func
0000000000004010 D global_var
                 U __libc_start_main@GLIBC_2.34
000000000000119b T main
0000000000004018 B uninit_var
```

Revision #11
Created 2026-08-21 03:05:08 UTC by Dat Nguyen
Updated 2026-09-03 02:01:17 UTC by Dat Nguyen